

Candidate Intelligence Platform

Redrob Hackathon v4 — Intelligent Candidate Discovery & Ranking

India Runs Data & AI Challenge · 2026-06-17

Problem

- Rank the top 100 of 100 000 candidates for a Senior AI Engineer role at a Series A AI-native talent platform.
- Scoring: $0.50 \cdot \text{NDCG@10} + 0.30 \cdot \text{NDCG@50} + 0.15 \cdot \text{MAP} + 0.05 \cdot \text{P@10}$.
- Hard constraints at ranking time: ≤ 5 min, 16 GB RAM, CPU only, no network.
- Three deliberate traps in the pool: keyword-stuffers, plain-language Tier 5s, and ~80 honeypots with subtly impossible profiles.
- The JD explicitly warns: 'the right answer is not find candidates whose skills section contains the most AI keywords.'

Data Profile

- Pool of 100 000 candidates, 487 MB JSONL.
- Top titles are dominated by non-tech roles: Mechanical Engineer, HR Manager, Content Writer, Business Analyst, Sales Exec, Customer Support, Accountant, Civil Engineer, Graphic Designer, Operations Manager, Project Manager, Marketing Manager.
- Software Engineer and friends are a minority; 'AI/ML' as an industry label is < 0.5 % of candidates.
- 75 % of candidates are in India; Noida + Pune have ~4 000 candidates each.
- Years-of-experience: median 6.8 yrs, mean 7.2 yrs (matches the 5-9 band).

Architecture

- Build phase (offline, network-allowed): BM25 index, dense index, feature store, LTR, LLM portraits.
- Rank phase (sandbox-reproducible): single Python process, no network, < 120 s wall on 16 GB CPU.
- Graceful degradation: if the dense index is missing, the pipeline falls back to BM25-only retrieval.
- Per-candidate behavioral scores (availability, positive, negative, honeypot) are precomputed in batch and looked up at rank time — no per-candidate Python calls in the inner loop.

Pipeline

- 1. Build the per-candidate `deep\_profile` text (career evidence first, skills second).
- 2. BM25 over the corpus + (optional) dense retrieval + RRF fusion.
- 3. Cross-encoder rerank of the top 500.
- 4. LightGBM LambdaRank scoring (trained with bucket groups to bypass the 10k-row limit).
- 5. Ensemble: $0.55 \cdot \sigma(\text{LTR}) + 0.20 \cdot \sigma(\text{CE}) + 0.10 \cdot \text{availability} + 0.10 \cdot \text{positive} - 0.10 \cdot \text{negative} - 0.20 \cdot \text{honeypot}$.
- 6. Strictly monotonic score calibration ($0.99 \rightarrow 0.20$ with jitter).
- 7. Look up pre-stored reasoning (LLM portrait, or feature-driven template fallback).

Trap-Aware Intelligence

- Honeypot risk = weighted sum of 7 rule signals: skill-proficiency-vs-duration, YOE-vs-career-sum, perfect-skill-list-with-non-tech-title, multiple current positions, expert-in-too-many-skills, all-skills-zero-endorsements, high-skill-count-no-career-evidence.
- JD negative filters (each contributes a weighted penalty): only-consulting, only-CV/robotics/speech, title-chaser, closed-source-only, LangChain-recent-only, no-NLP/IR-in-career, YOE-out-of-band.
- JD positive boosters: has-AI-career-evidence, shipped-ranking-or-search-at-scale, tier-1/2 education, Noida/Pune-or-relocate, GitHub/open-source, hybrid/onsite, sub-30-day notice, active-30-days.
- Honeypot signal is subtracted from the ensemble score, so a high-risk candidate can never climb into the top 100.

Hybrid Retrieval

- BM25 (rank_bm25) over the `deep\_profile` text catches literal vocabulary matches.
- Dense retrieval (BGE-small-en-v1.5, optional) catches semantic matches.
- Reciprocal Rank Fusion (k=60) combines the two; disagreement is informative.
- Cross-encoder (`ms-marco-MiniLM-L-6-v2`, 90 MB) reranks the top 500 → top 200.
- All four stages fit in < 5 s on 16 GB CPU; deterministic for the same index.

LTR + Ensemble

- LightGBM LambdaRank with 67 features (seniority, AI evidence, product-company count, location, JD positives/negatives, behavioral, honeypot, raw signals).
- Training labels = 0-4 proxy-relevance tier from JD-derived heuristics (AI evidence $\times$ seniority $\times$ location $\times$ product-company $\times$ behavior).
- Bucket groups (5 000 rows each) to satisfy the 10k-row-per-query limit while keeping one global ranking problem.
- Ensemble blends LTR, cross-encoder, behavioral availability, positive boosters, negative filters, and honeypot risk.

Benchmark

- Proxy NDCG@10 (full pool, 100 k) is hard to compute without ground truth; we use proxy-relevance on the dev split.
- 5-fold CV on the LTR yields a stable composite.
- Ablation on a 5 k dev split (in `reports/benchmark.md`):
 - random baseline — composite 0.00
 - YOE-only — composite ~ 0.05
 - industry_ai_ml only — composite ~ 0.10
 - skills_ai_count only — composite ~ 0.25
 - proxy_relevance (oracle) — composite ~ 0.65 (upper bound)
- LTR + ensemble closes ~ 80 % of the gap between the random and the proxy-oracle.

Results

- Top-10 dominated by ML Engineers, AI Engineers, Data Scientists, AI Research Engineers, Computer Vision Engineers — all in or near the 5-9 yrs band.
- Bottom of top-100 still includes some non-AI candidates (the JD accepts 10 great matches over 1 000 maybes), but the surface area is bounded by the strict monotonicity + honeypot subtraction.
- Submission validates against the official `validate\_submission.py`.
- Stage 3 reproduction: < 120 s on 16 GB CPU.

Lessons Learned

- The 5-min CPU budget is generous for ranking once the build is split out. Most of the time goes to building the BM25 index and the feature table — not the ranking step itself.
- LightGBM lambdarank has a 10k-row-per-query limit. Splitting into multiple 'queries' (5k-row buckets) keeps the same global ranking problem and is 50× faster than 1k-fold CV.
- Caching yaml config reads is a 30× speedup when scoring 100k candidates.
- Honeypot detection should be a score, not a hard filter. Subtracting the risk from the ensemble lets a perfect-on-paper candidate still be in the top 100 if all other signals are overwhelmingly positive (which, in this pool, never happens).
- When the LLM API is unreachable, a feature-driven template is the right fallback for the Stage 4 review — it cites a specific positive signal and an honest concern per candidate.

Future Work

- Replace bge-small-en with bge-m3 or NV-Embed-v2 in a one-time cloud build; same code path.
- Add a ColBERTv2 late-interaction rerank over the top 200 (5k rows × 128 tokens → 640k embeddings).
- Bootstrap a much larger LTR training set with active learning on the top-100 disagreements.
- Add a per-JD query rewriter (LLM, build-time) to expand the BM25 query with synonyms and related terms.
- Replace MiniLM with BGE-reranker-v2-m3 on a single A10G when GPU is available at build.

Competition Strategy

- Three-submission cap, no live leaderboard. We used a single best-effort submission to avoid spending attempts on partial experiments.
- Two attempts are kept in reserve in case the leaderboard shows a surprising ranking pattern after the reveal.
- Stage 4 (manual review) is where 80% of the tiebreaks will happen — reasoning quality, methodology coherence, git history authenticity.
- Final ranking emphasizes: (1) the trap-aware framing, (2) the career-vs-skills weighting, (3) the clean, deterministic, reproducible pipeline.

Thank You

- Repository: <https://github.com/arsat/india-runs-ranking>
- Reproducibility: ``python scripts/build_artifacts.py ...`` → ``python src/serving/rank.py ...``
- Stage 3 reproduction: < 120 s, 16 GB RAM, CPU only, no network.
- Questions welcome.